
Dossier de Conception – Application Web de Gestion d'Association (V1)

1. Introduction

1.1 Nom du projet

AssoManager (nom provisoire)

1.2 Objectif du projet

Développer une application web simple de **gestion d'association** permettant :

- la **gestion des utilisateurs** (rôles : **Admin**, **Membre**)
- l'**authentification** (inscription, connexion, déconnexion)
- le **paiement et le suivi des cotisations mensuelles**.

1.3 But du document

Ce dossier de conception traduit les exigences du Cahier des Charges en choix techniques et structures concrètes pour le développement :

- **Front-end : React (SPA)**
- **Back-end : Laravel (API REST)**

2. Architecture Générale

2.1 Vue d'ensemble

- **React (SPA)** : interface utilisateur, routing, formulaires, affichage des données
- **Laravel (API REST)** : authentification, logique métier, accès base de données

Interaction :

React ⇌ (Requêtes HTTP JSON / API) ⇌ Laravel ⇌ Base de données

2.2 Approche choisie

- SPA (Single Page Application) côté React
- API RESTful côté Laravel
- Authentification par **token** (Laravel Sanctum recommandé)

3. Modèle de Données (Structure Base de Données)

3.1 Table : users

- id (BIGINT, PK)
- first_name (VARCHAR)
- last_name (VARCHAR)
- email (VARCHAR, UNIQUE)
- password (VARCHAR)
- role (ENUM: ADMIN, MEMBER) (ou VARCHAR simple)
- status (ENUM: ACTIVE, INACTIVE) (ou BOOLEAN is_active)
- created_at (DATETIME)
- updated_at (DATETIME)

3.2 Table : contribution_payments

- id (BIGINT, PK)
- user_id (BIGINT, FK vers users)
- year (INT) — ex: 2026
- month (INT) — 1 à 12
- amount (DECIMAL ou INT)
- payment_date (DATE ou DATETIME)
- payment_method (ENUM: CASH, TRANSFER, OTHER)
- reference (VARCHAR, NULLABLE)
- created_at (DATETIME)
- updated_at (DATETIME)

3.3 Contraintes et règles métier

- **Unicité** : (user_id, year, month) UNIQUE
→ empêche 2 cotisations validées pour le même mois/année
- Un paiement appartient à **un seul membre**.

3.4 Relations (Eloquent)

- User **hasMany** ContributionPayment
- ContributionPayment **belongsTo** User

4. Conception de l'API Back-end (Laravel)

(Liste des endpoints + méthode + URL + description + besoin d'auth)

4.1 Auth

- **POST /api/register**
Créer un compte membre (role=MEMBER, status=ACTIVE). **(Auth: Non)**
- **POST /api/login**
Authentifier et retourner un token + user. **(Auth: Non)**
- **POST /api/logout**
Déconnecter (révoquer token). **(Auth: Oui)**

4.2 Profil (Membre connecté)

- **GET /api/me**
Retourne les infos du user connecté. **(Auth: Oui)**
- **PUT /api/me (optionnel V1)**
Mettre à jour profil (nom/prénom). **(Auth: Oui)**

4.3 Admin – Gestion des membres

- **GET /api/admin/users**
Liste des membres (pagination + search: name/email). **(Auth: Oui, ADMIN)**
- **GET /api/admin/users/{id}**
Détails d'un membre + paiements. **(Auth: Oui, ADMIN)**
- **PATCH /api/admin/users/{id}/status**
Activer/Désactiver un membre. **(Auth: Oui, ADMIN)**

4.4 Cotisations – Paiements

Côté Admin

- **POST /api/admin/users/{id}/payments**
Ajouter un paiement de cotisation pour un membre. **(Auth: Oui, ADMIN)**
- **GET /api/admin/payments**
Historique global + filtres (month/year/user). **(Auth: Oui, ADMIN)**

Côté Membre

- **GET /api/my/payments**
Historique personnel (filtre année). **(Auth: Oui)**

4.5 Format de réponses (simple)

- Succès : { success: true, data: ... }
- Erreur validation (422) : { message, errors }
- Non autorisé (401/403) : { message }

5. Structure du Front-end (React)

(Organisation pages/routes + composants + gestion état + navigation)

5.1 Organisation des dossiers (proposée)

- src/
 - api/ (axios config, services)
 - auth/ (context, guards)
 - components/ (Header, Sidebar, ProtectedRoute...)
 - pages/
 - public/ (Login, Register)
 - member/ (Dashboard, MyPayments, Profile)
 - admin/ (Dashboard, UsersList, UserDetails, AddPayment)
 - routes/ (router config)

5.2 Routes / Pages principales

Public

- /login
- /register

Membre

- /member/dashboard
- /member/payments
- /member/profile

Admin

- /admin/dashboard
- /admin/users
- /admin/users/

5.3 Composants majeurs

- Layout :
 - Header / TopBar
 - Sidebar (Admin/Membre)
 - ProtectedRoute (vérifie token)
 - RoleRoute (vérifie role)
- Auth :
 - LoginForm
 - RegisterForm
- Paiements :
 - PaymentsTable
 - AddPaymentModal/Form
- Utilitaires :
 - Loader
 - AlertMessage
 - Pagination

5.4 Gestion de l'état

- **AuthContext (Context API) :**
 - user, token, role
 - login(), logout()
- Stockage token : **localStorage** (V1 simple)
- Navigation : **react-router-dom**

7. Réflexions sur les exigences non fonctionnelles clés

(Sécurité, performance, éco-responsabilité)

7.1 Sécurité

- Auth par token (Sanctum)
- Middleware Laravel :
 - auth
 - adminOnly (role check)
- Validation Laravel (FormRequest) pour toutes entrées
- Hash des mots de passe (Laravel)
- Protection :
 - ORM Eloquent (évite SQL injection)
 - Côté React : validation + affichage erreurs API (sans exposer détails sensibles)

7.2 Performance

- Pagination sur listes (users/payments)
- Filtrage côté API (month/year/user)
- Éviter de “charger tout” au front
- Optimisation requêtes Laravel (eager loading : with('payments') au besoin)

7.3 Éco-responsabilité (actions concrètes)

- Pagination et filtres pour réduire volume de données
- Pas de refresh automatique inutile
- Limiter le nombre d’appels API (cache simple côté front si pertinent)
- Bundling Vite + lazy loading routes admin (optionnel)

8. Planification détaillée de la réalisation (1 semaine)

Jour 1

- Setup Laravel API + React Vite + DB
- Installation Sanctum + CORS + Router + Axios

Jour 2

- Migrations + modèles (users, contribution_payments)
- Seed admin par défaut
- Auth endpoints : register/login/logout

Jour 3

- Middleware role ADMIN
- Endpoints admin users (list/detail/status)
- Normalisation réponses erreurs

Jour 4

- Endpoints paiements (admin + membre)
- Contrainte unique (user_id, month, year)
- Tests manuels API via Postman/Insomnia

Jour 5

- Front Auth (login/register) + layout + context auth
- Protected routes + redirects (admin/member)

Jour 6

- Front Admin : users list/detail + ajout paiement
- Front Membre : dashboard + my payments

Jour 7

- Corrections, polish UI, gestion erreurs, loaders
- README + guide d'exécution

9. Points à clarifier / décisions à prendre pendant le développement

1. **Montant de cotisation**

- Fixe pour tous ? ou configurable (par admin) ?
- Pour V1 : libre saisie au paiement (simple)

2. **Gestion des mois "non payés"**

- Calcul dynamique : si aucun paiement enregistré → non payé
- Pas de table "cotisations attendues", uniquement paiements

3. **Désactivation membre**

- Interdire la connexion si status=INACTIVE (recommandé)
- Ou autoriser connexion mais bloquer certaines actions

4. **Stockage du token**

- V1 : localStorage (simple)
- V2 : cookie httpOnly (plus sécurisé)

5. **Reset password**

- Exclu V1, ajout possible ensuite